

Lecture 33: Kafka

1. Problem Kafka Solves

Modern distributed systems consist of **multiple independent services**.

Example: Food delivery system.

When an order is confirmed, multiple systems must react:

```
OrderConfirmed
  ↓
Restaurant Service
Delivery Assignment
Notification Service
Fraud Detection
Analytics
Recommendation Engine
```

Without Kafka, the Order Service must call every service directly:

```
Order Service → Restaurant Service
Order Service → Delivery Service
Order Service → Notification Service
Order Service → Analytics
Order Service → Fraud Detection
```

Problems with this architecture:

- Tight coupling between services
- Difficult to scale
- Hard to add new services
- Slow response times
- Failure in one service affects others
- Difficult to process historical data

Kafka solves this using **event streaming architecture**.

2. Event-Driven Architecture

In event-driven systems, services publish **events** when something happens.

Example:

```
OrderCreated  
OrderConfirmed  
OrderShipped  
OrderDelivered
```

Architecture:

```
Producer → Kafka → Consumers
```

Example:

```
Order Service → Kafka → Email Service
                    → Analytics
                    → Fraud Detection
                    → Delivery System
```

Kafka becomes the **central event pipeline**.

3. Database vs Kafka

These two serve different purposes.

Database

Stores the **current state**.

Example:

```
orders table
orderId → status = delivered
```

Purpose:

```
Query current state
Store business data
Source of truth
```

Kafka

Stores **events** describing changes.

Example:

```
OrderCreated  
OrderConfirmed  
OrderShipped
```

Purpose:

```
Broadcast events  
Build pipelines  
Enable replay  
Decouple services
```

4. Topics

A **Topic** is a stream of related events.

Example topics:

```
orders  
payments  
users  
inventory  
delivery
```

Example topic:

```
Topic: orders
```

Events:

```
OrderCreated  
OrderConfirmed  
OrderCancelled  
OrderShipped  
OrderDelivered
```

Topics represent **business domains**.

5. Partitions

Topics are split into **partitions** for scalability.

Example:

```
Topic: orders
```

```
Partition 0  
Partition 1  
Partition 2
```

Each partition is an **append-only log**.

Example:

Partition 0

Offset 0 → OrderCreated
Offset 1 → OrderConfirmed
Offset 2 → OrderShipped

Benefits:

Parallel processing
High throughput
Horizontal scaling

6. Ordering

Kafka guarantees **ordering inside a partition only**.

Example:

Partition 0

OrderCreated
OrderConfirmed
OrderShipped
OrderDelivered

Ordering is required when later events depend on earlier ones.

Example:

```
OrderConfirmed → must happen before → OrderShipped
```

If events go to different partitions, ordering is not guaranteed.

7. Partition Key

Kafka decides partition using a **key**.

Example:

```
key = orderId  
partition = hash(orderId) % number_of_partitions
```

This ensures:

```
All events for one order → same partition
```

So ordering is preserved.

8. Brokers

A **Broker** is a Kafka server.

Example cluster:

```
Broker 1  
Broker 2  
Broker 3
```

Each broker stores some partitions.

Example:

```
Broker1 → Partition 0  
Broker2 → Partition 1  
Broker3 → Partition 2
```

A cluster of brokers forms a **Kafka cluster**.

9. Producers

Producers send messages to Kafka.

Example:

```
Order Service  
Payment Service  
User Service
```

Producer sends events:

```
Order Service → orders topic
```

Producer chooses partition using:

```
partition = hash(key) % partitions
```

10. Consumers

Consumers read messages from Kafka.

Example consumers:

```
Email Service  
Analytics Service  
Fraud Detection  
Recommendation Engine
```

Consumers read messages sequentially from partitions.

11. Consumer Groups

Consumer groups allow scaling.

Example:

```
Email Service
```

If traffic increases:

```
Email Instance 1  
Email Instance 2  
Email Instance 3
```

All belong to one group:

```
email-service-group
```

Kafka distributes partitions:

```
Partition 0 → Email1  
Partition 1 → Email2  
Partition 2 → Email3
```

Rule:

```
One partition → one consumer in a group  
One consumer → multiple partitions allowed
```

12. Partition Assignment

Kafka uses a **Group Coordinator** to manage consumer groups.

Responsibilities:

```
Track consumers in group
Assign partitions
Handle rebalancing
```

Example:

```
Partitions: 4
Consumers: 3
```

Assignment:

```
Consumer1 → P0,P1
Consumer2 → P2
Consumer3 → P3
```

13. Offsets

Each message in a partition has a unique number.

Example:

```
Offset 0 → OrderCreated  
Offset 1 → OrderConfirmed  
Offset 2 → OrderShipped
```

Consumers track their reading position using offsets.

14. Offset Commit

After processing a message, the consumer commits the offset.

Example:

```
Processed up to offset = 2
```

Next read begins at:

```
Offset 3
```

Offsets are stored in Kafka in an internal topic:

```
__consumer_offsets
```

15. Replication

Kafka replicates partitions for fault tolerance.

Example:

```
Partition 0
```

```
Leader → Broker1
```

```
Follower → Broker2
```

```
Follower → Broker3
```

Writes go to the leader.

Followers replicate the data.

If leader fails:

```
Follower becomes new leader
```

16. KRaft (Metadata Management)

Kafka needs metadata like:

```
Topic partitions
```

```
Broker list
```

```
Leader assignments
```

Modern Kafka uses **KRaft (Kafka Raft)** to manage metadata.

Responsibilities:

```
Leader election  
Cluster metadata  
Broker health tracking
```

17. Hot Partition

Hot partition occurs when one partition receives most traffic.

Example:

```
Partition 0 → 10000 events/sec  
Partition 1 → 500 events/sec  
Partition 2 → 300 events/sec
```

Cause:

```
Poor key distribution
```

Example:

```
key = popular restaurantId
```

Solution:

```
Choose better key
Increase partitions
```

18. Kafka Performance Design

Kafka is extremely fast due to several design choices.

Sequential Writes

Kafka only appends data.

```
append → append → append
```

Sequential disk writes are very fast.

Page Cache

OS stores recently accessed disk data in RAM.

Flow:

```
Disk → Page Cache → Network
```

Most reads come from RAM instead of disk.

Batching

Producers send multiple messages together.

Example:

```
send 100 messages in one request
```

Benefits:

```
less network overhead  
fewer disk operations
```

Zero Copy

Kafka uses the Linux `sendfile()` system call.

Normal flow:

```
Disk → Kernel → App → Network
```

Kafka optimized flow:

Disk → Kernel → Network

This reduces memory copies and CPU usage.

19. Log Segments

Kafka splits partitions into smaller files called **segments**.

Example:

```
Segment1 → offsets 0-999  
Segment2 → offsets 1000-1999  
Segment3 → offsets 2000-2999
```

Kafka writes to the latest segment.

Old segments can be deleted easily.

20. Retention Policy

Kafka retains messages for a configured period.

Example:

```
retain for 7 days
```

Messages are deleted based on:

```
time  
size
```

Not based on consumption.

21. Replay

Because messages are retained, consumers can replay history.

Example:

```
Analytics wants last 3 days data
```

Consumer can start reading from older offsets.

This enables:

```
reprocessing  
machine learning training  
debugging  
data recovery
```

22. Log Compaction

Sometimes we want only the latest value for a key.

Example:

```
Offset0 → user1 = A  
Offset1 → user1 = B  
Offset2 → user1 = C
```

Compaction keeps:

```
user1 = C
```

Older records removed.

Used for:

```
state updates  
configuration  
user profiles
```

23. Delivery Guarantees

Kafka supports three delivery modes.

At-most-once

```
message delivered once or lost
```

No duplicates.

At-least-once (most common)

```
message delivered ≥ 1 time
```

Duplicates possible but no data loss.

Exactly-once

Kafka ensures message processed exactly once using:

```
idempotent producers  
transactions
```

24. Kafka Mental Model

Kafka is essentially a **distributed commit log**.

Architecture:

```
Producers  
  ↓  
Topics  
  ↓  
Partitions  
  ↓  
Brokers  
  ↓  
Consumer Groups  
  ↓  
Offsets
```

Kafka enables:

event streaming
data pipelines
real-time analytics
microservice communication
replayable logs

Final Summary

Kafka is best understood as:

A distributed append-only event log
that enables scalable, fault-tolerant data streaming.

Key strengths:

High throughput
Horizontal scaling
Fault tolerance
Event replay
Service decoupling

If you want, I can also create a **much deeper Kafka documentation (like a system design whitepaper)** covering:

- ISR (In-Sync Replicas)
- Leader election internals
- Producer acknowledgements
- Kafka transactions

- Rebalancing protocol
- Fetch requests
- Network protocol
- Exactly-once internals

That would be **closer to how Kafka actually works internally at LinkedIn / Uber scale.**



